

Multi-Agent Trade Document Pipeline

Product Requirement Document

1. Understanding Nova

What is Nova?

Nova is GoComet’s configurable agentic operations platform that automates enterprise workflows involving unstructured data and human decision-making. It has three layers: a YAML-based Workflow Orchestrator, an AI Agents layer (extraction, validation, decisions), and a No-Code App Builder. Traditional SaaS breaks on messy PDFs, emails, and customer-specific rules. Nova addresses this by enabling agent-driven systems that adapt per enterprise without rebuilding the product.

What is the FDE Model?

Forward Deployed Engineering means engineers work inside customer environments to understand real workflows, edge cases, and constraints. In Nova’s context, FDEs translate customer-specific reality into Nova configurations — because different customers use different document formats, approval chains, and validation rules. One product, many configurations.

What is a System of Outcomes?

System of Record stores data. System of Engagement helps users interact with software. Nova is a System of Outcomes — it completes operational tasks and drives measurable results. The shift: from “Where is the data?” to “Did the shipment clear correctly and faster?”. Operations teams care about shipment clearance speed and error reduction, not dashboards.

2. Problem Statement

Failure Modes

Failure Mode	Description	Example
Inconsistent extraction	Same field named differently across doc formats.	"COO" vs "Country of Origin" on the same invoice type.
Tribal knowledge	Validation rules live in operators' heads, not systems.	New CG hire approves wrong HS code; rules were undocumented.
Serial review	One-by-one doc review slows shipment processing.	5 attachments = 1+ hour of manual validation.
No auditability	Approval decisions are not traceable.	Customs dispute — no record of which version was approved.

Success — First 5 Minutes for a CG Operator

Upload a doc set, see all extracted fields with confidence scores and a validation verdict — no attachment opening. Clean shipments auto-clear. Mismatches surface with a pre-drafted amendment email. Uncertain fields never silently pass.

3. Personas + Jobs-to-be-Done

Persona 1 — Alice, Trade Ops Analyst (CG)

Receives 40–80 doc sets per day across customers with different validation rules. Goal: speed without sacrificing accuracy. She wants to touch only cases that need genuine judgment.

Persona 2 — Bob, Export Documentation Manager (SU)

Generates and submits shipment docs 15–30 times per week. Biggest frustration: vague rejection emails with no field-level specifics, forcing blind resubmission. 2–4 amendment cycles per shipment is his normal.

1. **JTBD 1 (CG)** — When a doc set arrives, I want to see extracted fields and a pass/fail without opening attachments, so that I can triage 30 shipments in the time it used to take me to review 5.
2. **JTBD 2 (CG)** — When the system flags a discrepancy, I want to see what failed, what was found, what was expected, and a draft amendment email, so that I can approve and send in one click.
3. **JTBD 3 (CG)** — When a new customer is onboarded, I want to define their rules once in a structured form, so that junior analysts can handle that customer on day one without shadowing.
4. **JTBD 4 (SU)** — When my submission is rejected, I want a field-level breakdown of every discrepancy, so that I can fix everything in one pass and stop the 3-cycle back-and-forth.
5. **JTBD 5 (SU)** — When I submit for a repeat customer, I want to know which fields they check most strictly, so that I can self-validate before submitting and avoid rejection entirely.

4. Agent Architecture

Why Three Agents?

A single prompt conflates three fundamentally different cognitive tasks: perceiving (reading the doc), judging (comparing against rules), and deciding (what to do next). Collapsing these means a hallucinated field value can silently pass validation because the validator never saw the raw document. Five agents over-decompose — splitting Extractor into “parse” and “extract” sub-agents creates handoff overhead without clarity. The test: does each agent have a distinct skill requirement, a distinct failure mode, and a distinct place to insert a human check? Three answer yes. The core principle: agent boundaries map to trust boundaries.

Agent Responsibilities

Agent	Role	Input	Output	Hard constraint
Extractor	Executor	Raw file bytes + doc type hint	ExtractionResult (see schema §6)	Absent field = null/0.0. No guessing.
Validator	Verifier	ExtractionResult + CustomerRuleSet	ValidationResult (see schema §6)	No access to raw doc. Uncertain fields must surface.
Router	Planner	ValidationResult + RoutingPolicy	RouterDecision (see schema §6)	Cannot re-interpret doc. Must explain decision.

Agent Communication

Agents pass structured JSON through a Postgres job store — not shared in-memory state. Shared memory lets agents pollute each other's context; structured handoff makes the interface explicit and testable. LangGraph manages state transitions and ensures agents only run when preconditions are met.

Crash Recovery

Every agent writes output to the job store before returning. LangGraph's checkpointing stores full graph state at each node transition. On restart, LangGraph replays from the last successful checkpoint — no duplicate LLM calls, predictable per-document cost.

5. LLM & Tooling Choices

Decided Stack

Layer	Decision	Rationale
API framework	FastAPI + Uvicorn	Async-native, Pydantic integration, auto OpenAPI docs. Matches LangGraph's async model.
Orchestration	LangGraph	Conditional edges, built-in checkpointing, async execution, visualisable graph. Custom orchestrator = 2–3 weeks of infra before a single agent runs.
Job store / state	PostgreSQL (via asyncpg)	Durable, queryable, supports the NL query layer. Redis would require a separate persistence layer for crash recovery.
Extractor LLM	GPT-4o (gpt-4o-2024-08-06)	Vision-capable, lower hallucination rate on blank fields vs Sonnet in internal tests on trade docs. Fallback: Claude 3.5 Sonnet if GPT-4o quota is unavailable.
Validator LLM	Claude 3 Haiku	Deterministic logic runs in Python. LLM handles fuzzy string matching only (entity name variants). Haiku: ~500ms, ~\$0.0003/doc.
Router LLM	Claude 3 Haiku	Draft amendment email is the only generative output. Haiku quality is sufficient; escalate to Sonnet only if draft quality fails review.
File storage	Local disk (dev) / S3 (prod)	Files stored at UPLOAD_DIR env var locally. S3_BUCKET env var switches to S3. Job record references file_path, not s3_key directly.
Frontend	React + Vite	Minimal UI per assignment requirement. Vite for fast dev loop. No SSR needed.
Query layer	DuckDB over Postgres JSONB	NL → SQL via Haiku. DuckDB allows analytical queries over exported JSONB without a separate data warehouse.

Structured Output & Function Calling

- Structured output (Pydantic / JSON mode): used for all agent outputs — `ExtractionResult`, `ValidationResult`, `RouterDecision`. Non-negotiable; free-text outputs cannot be parsed reliably at scale.
- Function calling: Extractor uses a tool schema to force field-by-field output with required confidence scores. Router uses a tool call to trigger `store_approved_document` — prevents storing by prose description.
- Avoided: inside Validator deterministic comparison logic. That runs in plain Python with unit tests — testable, fast, zero LLM cost.

6. Data Contracts & Schemas

All schemas are defined as Pydantic v2 models in `src/schemas/`. Every agent handoff uses these types. No free-text passing between agents.

ExtractionResult — Extractor → Validator

```
# src/schemas/extraction.py
from pydantic import BaseModel
from typing import Optional

class FieldExtraction(BaseModel):
    value: Optional[str]          # None if field not found in document
    confidence: float              # 0.0-1.0
    source_region: Optional[str]  # e.g. 'page 1, top-right table'

class ExtractionResult(BaseModel):
    job_id: str
    doc_type: str                  # 'BoL' | 'CommercialInvoice' | 'PackingList' |
    'CertificateOfOrigin'
    consignee_name: FieldExtraction
    hs_code: FieldExtraction
    port_of_loading: FieldExtraction
    port_of_discharge: FieldExtraction
    incoterms: FieldExtraction
    description_goods: FieldExtraction
    gross_weight: FieldExtraction
    invoice_number: FieldExtraction
    avg_confidence: float          # computed: mean across all fields
```

ValidationResult — Validator → Router

```
# src/schemas/validation.py
from pydantic import BaseModel
from typing import Literal, Optional

class FieldVerdict(BaseModel):
    status: Literal['match', 'mismatch', 'uncertain']
    found: Optional[str]
    expected: Optional[str]
    reason: str

class ValidationResult(BaseModel):
    job_id: str
    customer_id: str
    verdicts: dict[str, FieldVerdict]  # keyed by field name
    has_uncertain: bool                 # True if any field is uncertain
    mismatch_count: int
    uncertain_count: int
```

RouterDecision — Router → Storage + UI

```
# src/schemas/routing.py
from pydantic import BaseModel
from typing import Literal, Optional

class RouterDecision(BaseModel):
    job_id: str
    decision: Literal['approve', 'review', 'amend']
    reasoning: str # mandatory: agent must explain, not just label
    draft_message: Optional[str] # populated when decision == 'amend'
```

CustomerRuleSet — Validator input config

```
# src/schemas/rules.py
from pydantic import BaseModel
from typing import Optional, Literal

class FieldRule(BaseModel):
    expected_value: Optional[str] # exact match expected (e.g. HS code)
    allowed_values: Optional[list[str]] # any of these is acceptable
    match_type: Literal['exact', 'fuzzy', 'format_regex']
    pattern: Optional[str] # regex if match_type == 'format_regex'
    severity: Literal['hard', 'soft'] # hard = mismatch blocks approval
    required: bool

class CustomerRuleSet(BaseModel):
    customer_id: str
    customer_name: str
    rules: dict[str, FieldRule] # keyed by field name
```

CustomerRuleSet — Concrete Example (Acme Imports)

```
# configs/rules/acme_imports.json
{
  "customer_id": "ACME_001",
  "customer_name": "Acme Imports Pvt Ltd",
  "rules": {
    "consignee_name": {
      "expected_value": null,
      "allowed_values": ["Acme Imports Pvt Ltd", "Acme Imports Private Limited"],
      "match_type": "fuzzy",
      "pattern": null,
      "severity": "hard",
      "required": true
    },
    "hs_code": {
      "expected_value": null,
      "allowed_values": ["8471.30", "8471.41", "8471.49"],
      "match_type": "exact",
      "pattern": null,
      "severity": "hard",
      "required": true
    },
    "incoterms": {
      "expected_value": "CIF",
      "allowed_values": null,
      "match_type": "exact",
      "pattern": null,
      "severity": "soft",
      "required": true
    },
    "port_of_discharge": {
      "expected_value": "INNSA",
      "allowed_values": null,
      "match_type": "exact",
      "pattern": null,

```

```

    "severity": "hard",
    "required": true
  }
}
}

```

7. Database Schema

Single Postgres database. One table for the job store. Verified outputs stored as JSONB for flexible querying.

```

-- migrations/001_create_jobs.sql

CREATE TABLE jobs (
  job_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  status          TEXT NOT NULL DEFAULT 'pending',
  -- pending | extracting | validating | routing | complete | failed | loop_detected
  customer_id     TEXT NOT NULL,
  doc_type        TEXT NOT NULL,
  file_path       TEXT NOT NULL,          -- local path or S3 key
  extraction      JSONB,                  -- ExtractionResult
  validation      JSONB,                  -- ValidationResult
  decision        JSONB,                  -- RouterDecision
  retry_count     INT NOT NULL DEFAULT 0,
  token_usage     INT NOT NULL DEFAULT 0, -- cumulative tokens across all agents
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  completed_at    TIMESTAMPTZ
);

CREATE INDEX idx_jobs_status      ON jobs(status);
CREATE INDEX idx_jobs_customer   ON jobs(customer_id);
CREATE INDEX idx_jobs_created    ON jobs(created_at DESC);
CREATE INDEX idx_jobs_decision   ON jobs USING GIN ((decision->>'decision'));

```

8. API Contract

FastAPI application. Base URL: <http://localhost:8000>. All endpoints return JSON. Auth: Bearer token (env: `API_SECRET_KEY`). Full OpenAPI docs at `/docs`.

Method	Endpoint	Request body	Response
POST	<code>/jobs</code>	multipart/form-data: file, customer_id, doc_type	<code>{ job_id, status: "pending" }</code>
GET	<code>/jobs/{job_id}</code>	—	Full job record (status, extraction, validation, decision)
GET	<code>/jobs?customer_id=X&status=Y</code>	—	List of job summaries, paginated
POST	<code>/jobs/{job_id}/approve</code>	Optional: <code>{ operator_note }</code>	Updated job with status=complete
POST	<code>/query</code>	<code>{ question: "how many flagged this week?" }</code>	<code>{ answer, sql_used, rows }</code>

Method	Endpoint	Request body	Response
GET	/customers/{id}/rules	—	CustomerRuleSet for that customer
PUT	/customers/{id}/rules	CustomerRuleSet body	Updated rule set

9. Pipeline Sequence Flow

Step-by-step call order from upload to decision. All agent calls are async. UI polls /jobs/{job_id} for status updates.

6. **POST /jobs** → FastAPI writes job record to Postgres with status=pending, returns job_id.
7. **Background task fires** → LangGraph graph instantiated with job_id. Checkpointer attached to Postgres.
8. **Extractor node runs** → File fetched from UPLOAD_DIR. Sent to GPT-4o vision API with tool schema. ExtractionResult validated via Pydantic. Avg confidence computed. Job status → extracting.
9. **Confidence gate** → If avg_confidence < 0.60: skip Validator, set status=failed, escalate to human review. If ≥0.60: continue.
10. **Validator node runs** → CustomerRuleSet loaded from configs/rules/{customer_id}.json. Deterministic comparison runs in Python first. Haiku called only for fuzzy-match fields. ValidationResult written to job. Status → validating.
11. **Router node runs** → Haiku reads ValidationResult + RoutingPolicy. Produces RouterDecision with decision + reasoning + draft_message. If decision=approve: store_approved_document tool call fires. Status → routing.
12. **Job complete** → status=complete, completed_at set. UI polling picks up final state and renders result.
13. **On any agent failure** → Retry budget checked (max 2 per agent). If budget exceeded: status=failed, full trace logged. LangGraph recursion_limit=15 caps runaway loops.

10. Repo & Folder Structure

```

nova-trade-pipeline/
├── backend/
│   └── src/
│       ├── agents/
│       │   ├── extractor.py      # Extractor agent node
│       │   ├── validator.py     # Validator agent node
│       │   └── router.py        # Router agent node
│       ├── graph/
│       │   └── pipeline.py      # LangGraph graph definition + edges
│       ├── schemas/
│       │   ├── extraction.py    # ExtractionResult, FieldExtraction
│       │   ├── validation.py    # ValidationResult, FieldVerdict
│       │   ├── routing.py       # RouterDecision
│       │   └── rules.py         # CustomerRuleSet, FieldRule
│       ├── api/
│       │   ├── main.py          # FastAPI app, router mounts
│       │   └── routes/
│       │       ├── jobs.py      # /jobs endpoints
│       │       ├── query.py     # /query NL endpoint
│       │       └── customers.py # /customers endpoints
│       ├── db/
│       │   ├── connection.py    # asyncpg pool init
│       │   └── jobs_repo.py     # CRUD for jobs table

```

```

├── config.py          # Settings loaded from env vars
├── configs/
│   └── rules/
│       └── acme_imports.json  # One file per customer
├── migrations/
│   └── 001_create_jobs.sql
├── tests/
│   ├── test_extractor.py
│   ├── test_validator.py
│   └── test_router.py
├── uploads/          # UPLOAD_DIR (dev only)
├── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── UploadPanel.jsx
│   │   │   ├── ExtractionView.jsx
│   │   │   ├── ValidationView.jsx
│   │   │   ├── DecisionPanel.jsx
│   │   │   └── QueryBox.jsx
│   │   └── App.jsx
│   └── package.json
├── sample_docs/
│   ├── clean_bol.pdf
│   └── messy_invoice_scan.pdf
└── README.md

```

11. Environment Variables

All config is loaded from environment variables via `src/config.py` (using `pydantic-settings`). Copy `.env.example` to `.env` and fill in values before running.

Variable	Required	Example / Default	Description
OPENAI_API_KEY	Yes	sk-...	GPT-4o for Extractor agent
ANTHROPIC_API_KEY	Yes	sk-ant-...	Claude Haiku for Validator & Router
DATABASE_URL	Yes	postgresql+asyncpg://user:pass@localhost:5432/nova	Postgres connection string
UPLOAD_DIR	Yes	./uploads	Local file storage dir (dev). Set S3_BUCKET for prod.
S3_BUCKET	No	nova-trade-docs	If set, files stored in S3 instead of local disk
AWS_ACCESS_KEY_ID	No	—	Required if S3_BUCKET is set

Variable	Required	Example / Default	Description
AWS_SECRET_ACCESS_KEY	No	—	Required if S3_BUCKET is set
API_SECRET_KEY	Yes	any-long-random-string	Bearer token for API auth
RULES_DIR	No	./configs/rules	Directory containing customer rule JSON files
MAX_RETRIES_PER_AGENT	No	2	Max LLM retries per agent before force-fail
MAX_TOKENS_PER_JOB	No	20000	Token budget ceiling; job fails + alerts if exceeded
LANGGRAPH_RECURSION_LIMIT	No	15	LangGraph step cap to prevent infinite loops
CONFIDENCE_ESCALATE_THRESHOLD	No	0.60	Avg confidence below this escalates to human before Validator
CONFIDENCE_LOW_THRESHOLD	No	0.85	Field confidence below this tagged low_confidence

12. Trust, Failure Handling & Evals

Hallucination Prevention

Extractor prompt instruction: “If you cannot locate a field value in the document, return null with confidence 0.0. Do not infer, estimate, or fill from context.” Instruction alone is insufficient. Enforcement: confidence gating + post-extraction format checks (regex / checksum) for hard fields (HS codes, container IDs, certificate numbers). Any value failing the format check is immediately marked uncertain regardless of model confidence.

Confidence Threshold Ladder

Confidence range	Validator behaviour	Router outcome
≥0.85	Field passes normally	Eligible for auto-approve
0.60–0.85	Passes tagged low_confidence → Validator marks uncertain	Mandatory human review
<0.60 (field)	Does not enter comparison → marked uncertain	Mandatory human review
Avg <0.60 (doc)	Entire doc escalated before Validator runs	Human review + raw doc attached

Hard rule: Router cannot auto-approve any document with one or more uncertain fields. This is enforced in the RouterDecision schema — the approve branch only executes when has_uncertain=false and mismatch_count=0.

Loop & Cost Controls

- **Retry budget:** 2 retries per agent × 3 agents = 9 max LLM calls. Enforced in jobs_repo.py retry_count check, not in agent logic.
- **LangGraph recursion_limit:** 15 steps. Any job hitting the limit: status=loop_detected, full trace logged, queued for manual inspection.
- **Token ceiling:** MAX_TOKENS_PER_JOB (default 20,000 ≈ \$0.003 at Haiku pricing). Catches 200-page docs submitted accidentally.

Evals

- **Offline:** Golden dataset of 50–100 manually verified docs. Metric: % of fields with correct value AND confidence ≥0.75 (trusted extraction rate). Run on every model upgrade, prompt change, or PDF pre-processing change.
- **Online:** Operator override rate — % of Router decisions overridden by CG. Target: <5% within 4 weeks. >10% at 4 weeks = rule config is incomplete, not a model problem.

13. Metrics & Success Criteria

North-Star Metric

Percentage of shipments auto-resolved without a CG operator touching them. Pilot deployment target: 80%. If this number goes up, the system is working. If it stagnates, the failure is in extraction accuracy, rule coverage, router calibration, or operator trust.

Supporting Metrics

Metric	Category	Target	Why it matters
Extraction accuracy (correct at conf ≥0.75)	Agent quality	>92%	Low here = everything downstream wrong
p95 pipeline latency	System health	<90s	Must finish before CG needs the result
CG operator override rate	Trust	<5%	High = rules misconfigured or model untrusted
False approvals reaching customer	Safety	0	Non-negotiable; one breaks the pilot

Metric	Category	Target	Why it matters
Avg amendment cycles per shipment	Business outcome	<1.5×	Direct SU time savings, visible to customer
Cost per document (all LLM calls)	System health	<\$0.08	Margin model only works below this ceiling
Pipeline job completion rate	System health	>99%	Silent job loss is invisible to users

Go / No-Go — 2-Week Pilot

70% is the minimum viable bar; 80% is the deployment target. The 10-point buffer accounts for edge-case docs in week 1 without codified rules.

Go — all must be met by day 14:

- Auto-resolution rate ≥70%
- Zero false approvals
- CG operator override rate <10%
- p95 latency <3 minutes

No-Go — any triggers an immediate stop:

- Auto-resolution rate <50% at day 14: rule set underconfigured. Do not expand.
- Any false approval reaching a customer: hard stop. Full audit required before resuming.
- Override rate >20% at day 14: rule config does not reflect real customer requirements. Re-examine before continuing.

14. What's Next

Four gaps separate a working demo from a tool CG operators would miss. Each is a prerequisite for the one following it.

1. Email trigger & inbox wiring. The pipeline currently requires manual upload. The moment SU sends their email, the pipeline should fire. This is the difference between a proof of concept and a product. Nothing else matters until this is solved.

2. Cross-document consistency validation. A shipment arrives as BoL + CI + Packing List. The current Validator checks each against customer rules but not whether consignee name on the BoL matches the CI. Cross-document agreement is where the highest-frequency real-world discrepancies live.

3. Rule configuration UI for CG team leads. Adding a new customer currently means editing a JSON file and deploying code — the same tribal-knowledge problem one layer higher. A form-based UI makes the second customer take one day instead of two weeks.

4. Extraction quality feedback loop. Every operator correction is currently discarded. Storing it as a labelled example and using it to improve the Extractor turns each correction into training data. Without this, quality hits a ceiling.